

Cryptographic Scenarios

Beshir Said

- 1) Alice and Bob would use Diffie-Hellman to agree on a shared secret from which they would derive a shared symmetric key, K . Alice encrypts her long message using $AES(K, M)$ and sends the ciphertext to Bob. Then, Bob can decrypt using $AES_D(K, CP)$. This protocol works because Diffie-Hellman ensures that only Bob and Alice know the shared key, and AES provides the encryption confidentiality, so Eve cannot read the message.
- 2) Alice and Bob would use Diffie-Hellman to agree on a shared secret from which they would derive a shared symmetric key, K . Alice would then compute a Hash(SHA-256) where $h = H(\text{message})$ and then sign it with her private key: $\text{sig} = E(S_A, h)$. She sends the (M, sig) to Bob and Bob would compute the $H(\text{message})$ again and be able to check that $E(P_A, \text{sig}) = H(\text{message})$. This protocol works because the hash would detect any change to the message and the signature would prove authenticity since only Alice can produce the valid signature with S_A , therefore Mal can't modify the message without Bob not realizing the invalidation of the signature.
- 3) Alice and Bob would use Diffie-Hellman to agree on a shared secret from which they would derive a shared symmetric key, K . Alice would then compute a Hash(SHA-256) where $h = H(\text{message})$ and then sign it with her private key: $\text{sig} = E(S_A, h)$. She would then encrypt both, $C = AES(K, \text{message} \parallel \text{sig})$ and send C to Bob, who would decrypt to get $(\text{message}, \text{sig})$ and compute $H(\text{message})$ and verify with the P_A . This protocol works because AES allows for confidentiality so Eve can't read the contents, and the digital signature and hash ensure that Bob can verify that Alice is the true sender and that the contents haven't been altered.

4) Alice's repudiation claims against the Bob's allegations:

1) An attacker caused Bob to verify the signature against the wrong public key by getting a forged certificate for Alice. A certificate is public and, by itself, it doesn't prove the one who sent it owns the original private key, meaning just because you got a certificate with Alice's name doesn't mean it came from Alice. If I was the judge, I would ask Bob if he did any certificate checking. If not I would find this claim plausible; if he did, I'd find it unlikely.

2) The attacker got access to Alice's computer and stole Alice's private key, so someone else signed C. If I was the judge, I would find this claim somewhat reasonable but would need some video evidence or some log evidence to back this.

3) Alice signed a different byte sequence and Bob's C is a look-alike whose bytes differ from what Alice signed because the process they used didn't make the bytes unambiguous. This is important because without it, Alice can say I signed C while Bob shows a visually identical but byte-different version that looked the same to Alice during the signing. If I was a judge, I would be highly convinced with this claim unless there was an agreement on a form of standardized signing.

5) The Sig_CA is the Certificate Authority's signature over the hash of the data. The formula: $\text{Sig_CA} = E(S_CA, h(\text{data}))$ where data is inside the certificate.

6) No, the certificate is a publicly available document. Alice would need to send Bob a "challenge x" to prove that Bob has the correct private key that goes with the public key. Alice would send a hashed form of data. Bob signs it and sends the signature back where $\text{sig} = (E(S_B), H(x))$. Alice would then verify with Bob's public key (P_B) from the certificate and if it matches, Bob must have S_B .

7) One way is Mal stealing Bob's private key (S_B). Mal can therefore impersonate Bob freely without Alice knowing. Another way is Mal sending or installing a malware on Alice's computer that disables certificate validation, so that any certificate is accepted.